In [1]:
```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.linear_model import LogisticRegression
```

In [2]:
```
df = pd.read_csv('h1.csv')
```

In [3]:
```
df.head()
```

Out[3]:

	id	age	sex	dataset	cp	trestbps	chol	fbs	restecg	th
0	1	63	Male	Cleveland	typical angina	145.0	233.0	True	lv hypertrophy	1
1	2	67	Male	Cleveland	asymptomatic	160.0	286.0	False	lv hypertrophy	1
2	3	67	Male	Cleveland	asymptomatic	120.0	229.0	False	lv hypertrophy	1
3	4	37	Male	Cleveland	non-anginal	130.0	250.0	False	normal	1
4	5	41	Female	Cleveland	atypical angina	130.0	204.0	False	lv hypertrophy	1

In [4]:
```
df.shape
```

Out[4]:
```
(920, 16)
```

In [5]:
```
df.drop_duplicates(inplace = True)
```

In [6]:
```
df.shape
```

Out[6]:
```
(920, 16)
```

In [7]:
```
print("Datatypes: \n", df.dtypes)
```

```
Datatypes:
 id            int64
age           int64
sex          object
dataset      object
cp           object
trestbps    float64
chol        float64
fbs          object
restecg      object
thalch      float64
exang        object
oldpeak     float64
slope        object
ca          float64
thal         object
num           int64
dtype: object
```

In [8]:
```
#performing data cleaning and error correcting
pd.set_option('future.no_silent_downcasting',True)
```

In [9]:
```
mean_columns = ['trestbps' , 'chol' , 'thalch' , 'oldpeak' , 'ca']
mode_columns = ['fbs' , 'restecg' , 'exang' , 'slope' , 'thal']
```

In [10]:
```
for col in mean_columns:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].mean())
#we performed a command where all the numeric columns where null values w
#were filled with the mean
```

In [11]:
```
for col in mode_columns:
    if col in df.columns:
        df[col] = df[col].fillna(df[col].mode()[0])
#we have done the same as before but this time for categorical values
```

In [12]:
```
df = df.infer_objects(copy = False)
#converts columns that may be an object into the nearest possible datatyp
#library might work the best
```

In [13]:
```
print("The number of missing values in each column is : \n", df.isna().su
```

```
The number of missing values in each column is :
 id          0
age          0
sex          0
dataset      0
cp           0
trestbps     0
chol         0
fbs          0
restecg      0
thalch       0
exang        0
oldpeak      0
slope        0
ca           0
thal         0
num          0
dtype: int64
```

In [14]:
```
# performing integration: we have added the column hospital_id for better
df['hospital_id'] = np.random.randint(100,105, size = len(df))
```

In [15]:
```
df
```

Out[15]:

	id	age	sex	dataset	cp	trestbps	chol	fbs	rest
0	1	63	Male	Cleveland	typical angina	145.000000	233.0	True	hypertro
1	2	67	Male	Cleveland	asymptomatic	160.000000	286.0	False	hypertro
2	3	67	Male	Cleveland	asymptomatic	120.000000	229.0	False	hypertro
3	4	37	Male	Cleveland	non-anginal	130.000000	250.0	False	nor
4	5	41	Female	Cleveland	atypical angina	130.000000	204.0	False	hypertro
...	...	...	...	...	...	...	...	...	
915	916	54	Female	VA Long Beach	asymptomatic	127.000000	333.0	True	abnorma
916	917	62	Male	VA Long Beach	typical angina	132.132404	139.0	False	abnorma
917	918	55	Male	VA Long Beach	asymptomatic	122.000000	223.0	True	abnorma
918	919	58	Male	VA Long Beach	asymptomatic	132.132404	385.0	True	hypertro
919	920	62	Male	VA Long Beach	atypical angina	120.000000	254.0	False	hypertro

920 rows × 17 columns

In [16]:
```
df1 = df[['age' , 'cp' , 'chol' , 'thalch']]
df2 = df[['exang' , 'slope' , 'num']]
```

In [17]:
```
merged_df = pd.concat([df1,df2], axis = 1)
```

In [18]:
```
print("Shape of the merged dataframe: ", merged_df.shape)
print("Columns: ", merged_df.columns.tolist())
```

```
Shape of the merged dataframe:  (920, 7)
Columns:  ['age', 'cp', 'chol', 'thalch', 'exang', 'slope', 'num']
```

In [19]:
```
#removal of outliers using the IQR method

def mark_outliers(column):
    #we check if the coluns is numeric(integer, unsigned, float, complex)
    if column.dtype.kind in 'iufc':
        Q1 = column.quantile(0.25)
        Q3 = column.quantile(0.75)
        IQR = Q3 - Q1
        threshold = 1.5 * IQR

        #boolean mask for the outliers
        outlier_mask = (column < Q1 - threshold) | (column > Q3 + thresho

        #return only outliers, anything other than outliers become NaN va
        return column.where(outlier_mask)
    else:
        #non-numeric columns to be returned as is
        return column
```

In [20]:
```
numeric_columns = ['age', 'trestbps' , 'chol' , 'thalch' , 'oldpeak' , 'c
```

In [21]:
```
for col in numeric_columns:
    if col in df.columns:
        df[f'{col}_outliers'] = mark_outliers(df[col])
#the above will create new columns for marked outliers
```

In [22]:
```
outlier_columns = [f'{col}_outliers' for col in numeric_columns]
```

In [23]:
```
outliers_df = df[outlier_columns].dropna(how = 'all')
```

In [24]:
```
print("Rows with outliers: \n", outliers_df)
```

```
Rows with outliers:
      age_outliers  trestbps_outliers  chol_outliers  thalch_outliers  \
0              NaN                NaN            NaN              NaN
1              NaN                NaN            NaN              NaN
2              NaN                NaN            NaN              NaN
3              NaN                NaN            NaN              NaN
4              NaN                NaN            NaN              NaN
..             ...                ...            ...              ...
841            NaN                NaN            NaN              NaN
854            NaN              172.0            NaN              NaN
863            NaN                NaN            NaN              NaN
889            NaN              180.0            NaN              NaN
896            NaN              190.0            NaN              NaN

     oldpeak_outliers  ca_outliers
0                 NaN          0.0
1                 NaN          3.0
2                 NaN          2.0
3                 NaN          0.0
4                 NaN          0.0
..                ...          ...
841               4.0          NaN
854               NaN          NaN
863               4.0          NaN
889               NaN          NaN
896               NaN          NaN

[501 rows x 6 columns]
```

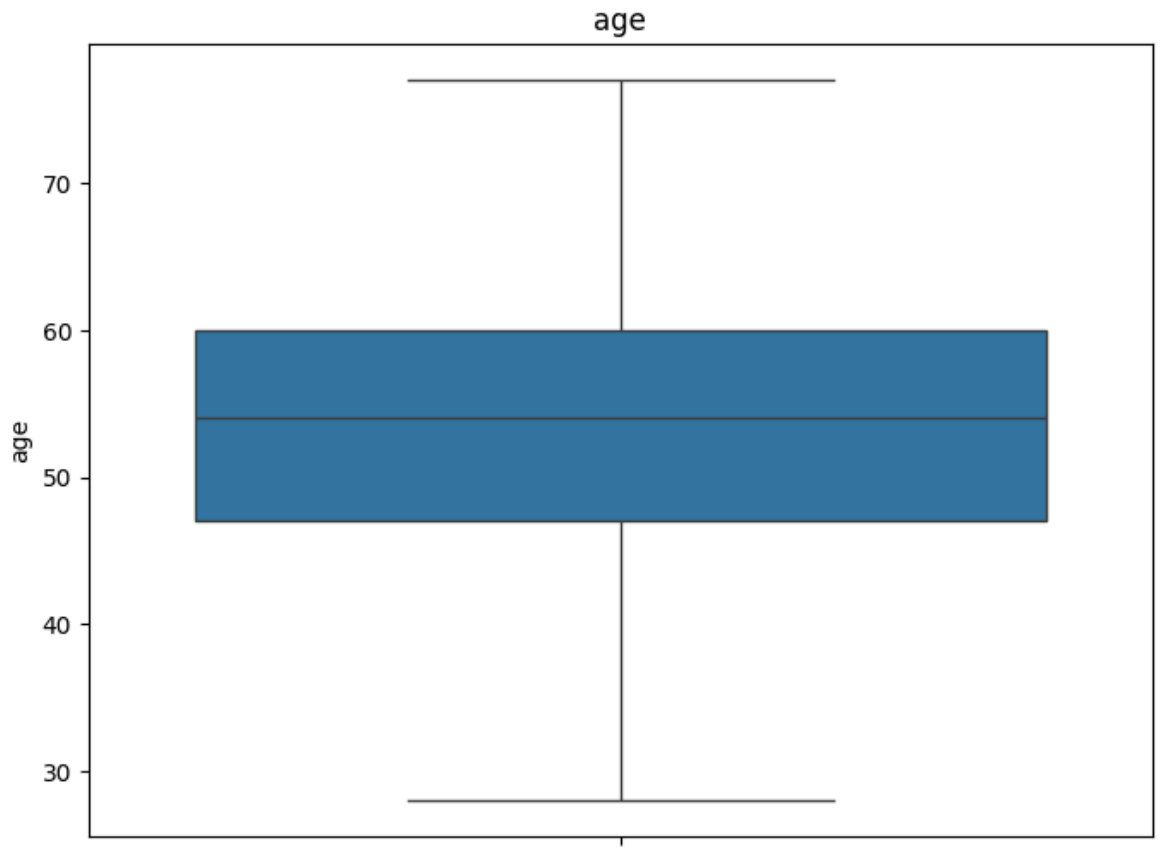
In [25]:
```python
# to remove the outliers

def remove_outliers(df, column):
    # check if the column is numeric
    if df[column].dtype.kind in 'iufc':
        Q1 = df[column].quantile(0.25)
        Q3 = df[column].quantile(0.75)
        IQR = Q3 - Q1

        lower_bound = Q1 - 1.5 * IQR
        upper_bound = Q3 + 1.5 * IQR

        df = df[(df[column] >= lower_bound) & (df[column] <= upper_bound)]

    return df
```

In [26]:
```python
df_clean = df.copy()  # safer than df = df

for col in numeric_columns:
    if col in df_clean.columns:
        df_clean = remove_outliers(df_clean, col)

print("Cleaned Dataframe without the outliers:\n", df_clean)
```

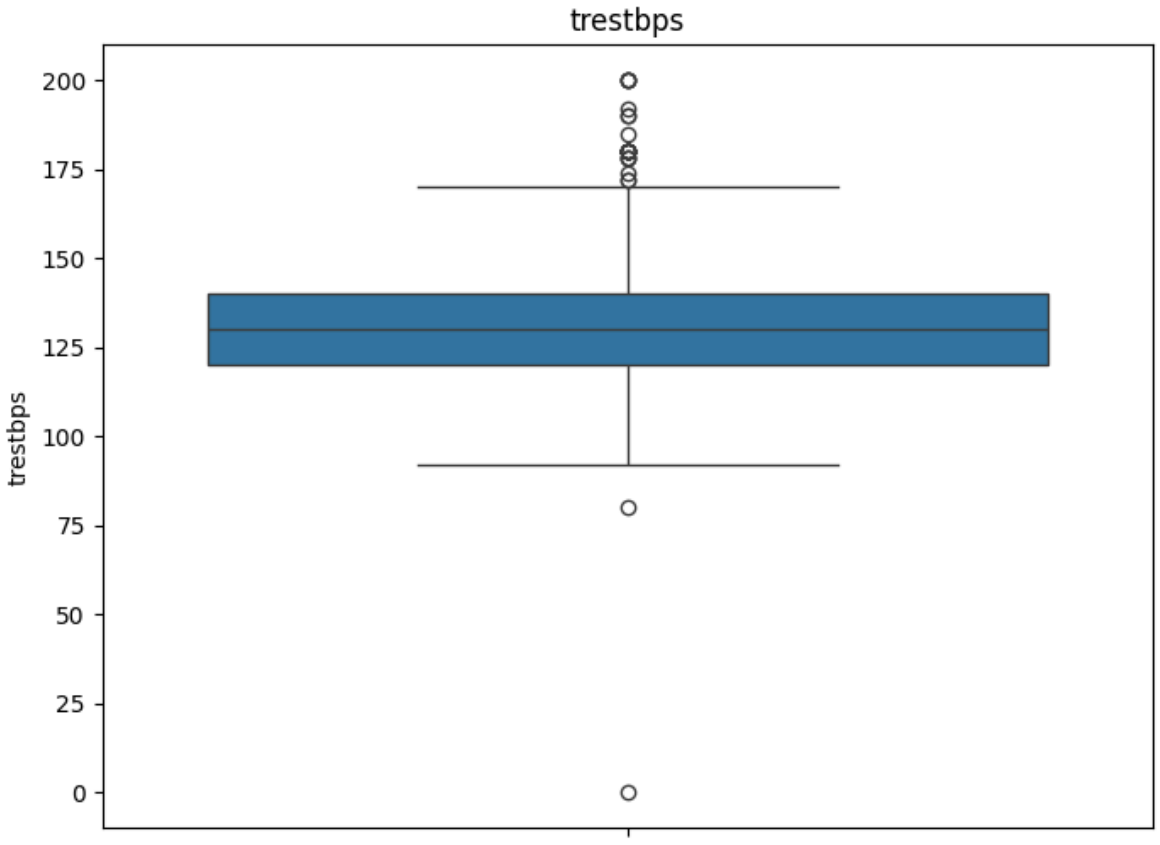
```
Cleaned Dataframe without the outliers:
       id  age     sex        dataset               cp    trestbps   chol
\
166  167   52    Male      Cleveland      non-anginal  138.000000  223.0
192  193   43    Male      Cleveland     asymptomatic  132.000000  247.0
287  288   58    Male      Cleveland  atypical angina  125.000000  220.0
302  303   38    Male      Cleveland      non-anginal  138.000000  175.0
303  304   28    Male      Cleveland  atypical angina  130.000000  132.0
..   ...  ...     ...            ...              ...         ...    ...
915  916   54  Female  VA Long Beach     asymptomatic  127.000000  333.0
916  917   62    Male  VA Long Beach   typical angina  132.132404  139.0
917  918   55    Male  VA Long Beach     asymptomatic  122.000000  223.0
918  919   58    Male  VA Long Beach     asymptomatic  132.132404  385.0
919  920   62    Male  VA Long Beach  atypical angina  120.000000  254.0

       fbs           restecg      thalch  ...        ca               thal
\
166  False            normal  169.000000  ...  0.676375             normal
192   True    lv hypertrophy  143.000000  ...  0.676375  reversable defect
287  False            normal  144.000000  ...  0.676375  reversable defect
302  False            normal  173.000000  ...  0.676375             normal
303  False    lv hypertrophy  185.000000  ...  0.676375             normal
..     ...               ...         ...  ...       ...                ...
915   True  st-t abnormality  154.000000  ...  0.676375             normal
916  False  st-t abnormality  137.545665  ...  0.676375             normal
917   True  st-t abnormality  100.000000  ...  0.676375       fixed defect
918   True    lv hypertrophy  137.545665  ...  0.676375             normal
919  False    lv hypertrophy   93.000000  ...  0.676375             normal

    num  hospital_id age_outliers  trestbps_outliers  chol_outliers  \
166   0          104          NaN                NaN            NaN
192   1          102          NaN                NaN            NaN
287   0          102          NaN                NaN            NaN
302   0          101          NaN                NaN            NaN
303   0          102          NaN                NaN            NaN
..   ..          ...          ...                ...            ...
915   1          101          NaN                NaN            NaN
916   0          103          NaN                NaN            NaN
917   2          104          NaN                NaN            NaN
918   0          103          NaN                NaN            NaN
919   1          101          NaN                NaN            NaN

     thalch_outliers  oldpeak_outliers  ca_outliers
166              NaN               NaN          NaN
192              NaN               NaN          NaN
287              NaN               NaN          NaN
302              NaN               NaN          NaN
303              NaN               NaN          NaN
..               ...               ...          ...
915              NaN               NaN          NaN
916              NaN               NaN          NaN
917              NaN               NaN          NaN
918              NaN               NaN          NaN
919              NaN               NaN          NaN

[419 rows x 23 columns]
```

In [27]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(8, 6))
sns.boxplot(data=df['age'])
```

```
plt.title('age')
plt.show()
```



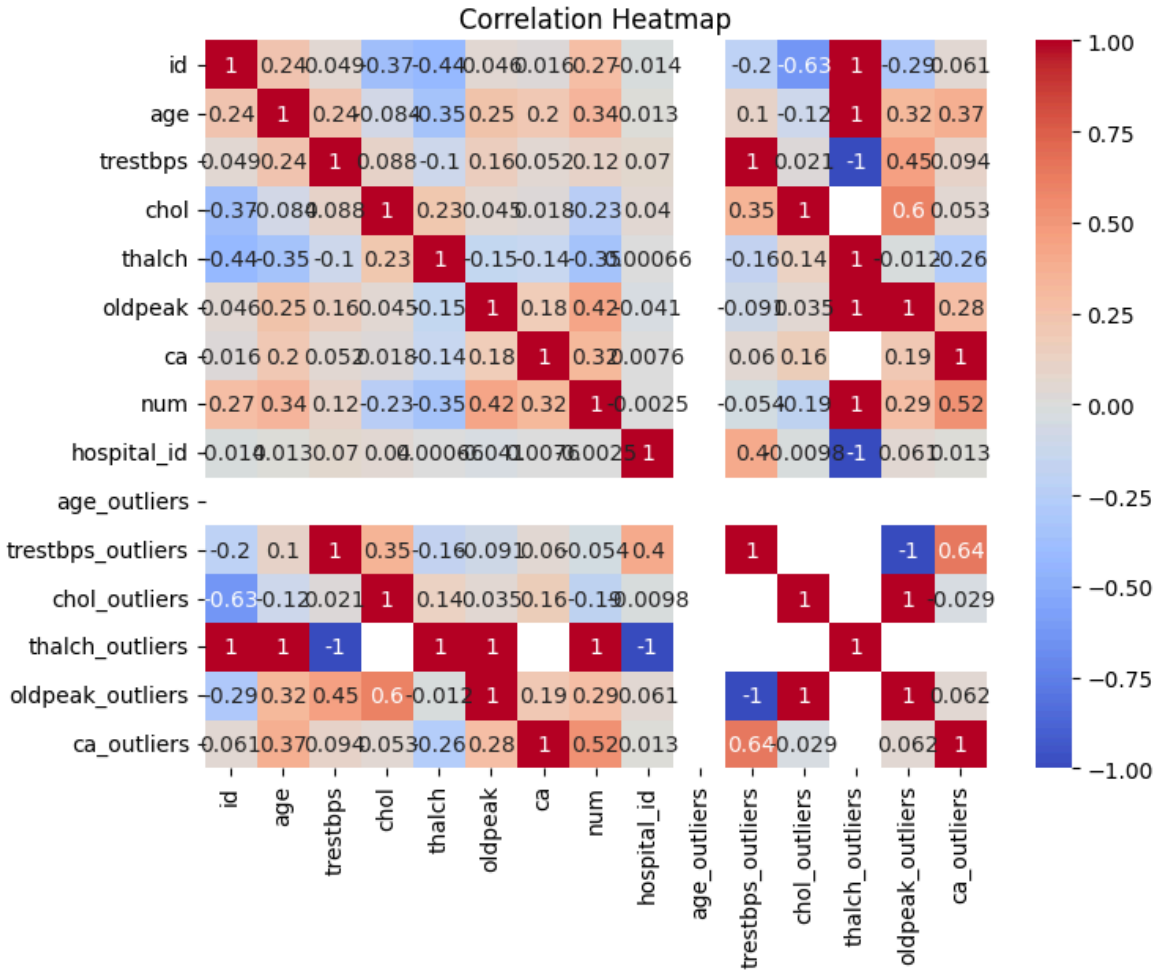
In [28]:
```
plt.figure(figsize=(8, 6))
sns.boxplot(data=df['trestbps'])
plt.title('trestbps')
plt.show()
```



In [29]:
```
#import LabelEncoder to convert categorical text data into numbers
from sklearn.preprocessing import LabelEncoder

#select only numeric columns from the dataframe
numeric_df = df.select_dtypes(include = np.number)

#calculate correlation of each column  with 'num' and remove 'num' itself
correlations = numeric_df.corr()['num'].drop('num')
#print correlation values with the target column
print("Correlation with the target: \n", correlations)
print()
```

```
Correlation with the target:
 id                   0.273552
age                  0.339596
trestbps             0.116225
chol                -0.228238
thalch              -0.351055
oldpeak              0.421907
ca                   0.321404
hospital_id         -0.002544
age_outliers              NaN
trestbps_outliers   -0.053648
chol_outliers       -0.192885
thalch_outliers      1.000000
oldpeak_outliers     0.286228
ca_outliers          0.516216
Name: num, dtype: float64
```

In [30]:
```
plt.figure(figsize=(8, 6))
sns.heatmap(numeric_df.corr(), annot=True, cmap ='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



In [31]:
```
from sklearn.preprocessing import LabelEncoder
```

In [32]:
```
df1 = df.copy() #copying the original frames

# initialize encoders
label_encoder = LabelEncoder()
scaler = StandardScaler()
```

In [33]:
```
#define columns
categorical_cols = ['sex', 'dataset', 'cp', 'restecg', 'slope', 'thal']
numeric_cols = ['age', 'trestbps', 'chol', 'thalch', 'oldpeak', 'ca']
```

In [34]:
```
#encode categorical columns
for col in categorical_cols:
    if col in df1.columns:
        df1[col] = label_encoder.fit_transform(df1[col].astype(str))
```

In [35]:
```
#handle missing values in numeric columns
df1[numeric_cols] = df1[numeric_cols].fillna(df1[numeric_cols].mean())
```

In [36]:
```
df1[numeric_cols] = scaler.fit_transform(df1[numeric_cols]) #scale numeri
```

In [37]:
```
print("Processed DataFrame:")
print(df1.head())
```

```
Processed DataFrame:
   id       age  sex  dataset  cp  trestbps      chol    fbs  restecg  \
0   1  1.007386    1        0   3  0.698041  0.311021   True        0
1   2  1.432034    1        0   0  1.511761  0.797713  False        0
2   3  1.432034    1        0   0 -0.658158  0.274289  False        0
3   4 -1.752828    1        0   2 -0.115679  0.467130  False        1
4   5 -1.328180    0        0   1 -0.115679  0.044717  False        0

     thalch  ...        ca  thal  num  hospital_id  age_outliers  \
0  0.495698  ... -1.249371     0    0          102           NaN
1 -1.175955  ...  4.292099     1    2          103           NaN
2 -0.340128  ...  2.444942     2    1          102           NaN
3  1.968345  ... -1.249371     1    0          100           NaN
4  1.371326  ... -1.249371     1    0          104           NaN

   trestbps_outliers  chol_outliers  thalch_outliers  oldpeak_outliers  \
0                NaN            NaN              NaN               NaN
1                NaN            NaN              NaN               NaN
2                NaN            NaN              NaN               NaN
3                NaN            NaN              NaN               NaN
4                NaN            NaN              NaN               NaN

   ca_outliers
0          0.0
1          3.0
2          2.0
3          0.0
4          0.0

[5 rows x 23 columns]
```

In [38]:
```
df['thalch'].replace({
    'fixeddefect': 'fixed_defect',
    'reversabledefect': 'reversable_defect'
})
```

Out[38]:
```
0      150.000000
1      108.000000
2      129.000000
3      187.000000
4      172.000000
          ...
915    154.000000
916    137.545665
917    100.000000
918    137.545665
919     93.000000
Name: thalch, Length: 920, dtype: float64
```

In [39]:
```
df['cp'] = df['cp'].replace({
    'typicalangina': 'typical_angina',
    'atypicalangina': 'atypical_angina'
})
```

In [40]:
```
df['restecg'] = df['restecg'].replace({
    'normal': 'normal',
    'st-t abnormality': 'ST-T_wave_abnormality',
    'lv hypertrophy': 'left_ventricular_hypertrophy'
})
```

In [41]:
```
data_1 = df[['age', 'sex', 'cp', 'dataset', 'trestbps', 'chol', 'fbs',
             'restecg', 'thalch', 'exang', 'oldpeak', 'slope', 'ca', 'tha

#we have taken out the columns which are neccessary to us
```

In [42]:
```
data_1['target'] = (df['num'] > 0).astype(int)

data_1['sex'] = (df['sex'] == 'Male').astype(int)

data_1['fbs'] = df['fbs'].astype(int)
data_1['exang'] = df['exang'].astype(int)
```

In [43]:
```
data_1.columns = [
    'age', 'sex', 'chest_pain_type', 'country', 'resting_blood_pressure',
    'cholesterol', 'fasting_blood_sugar', 'restecg',
    'max_heart_rate_achieved', 'exercise_induced_angina',
    'st_depression', 'st_slope_type', 'num_major_vessels',
    'thalassemia_type', 'target'
]
```

In [44]:
```
data_1.head()
```

Out[44]:

	age	sex	chest_pain_type	country	resting_blood_pressure	cholesterol	fasti
0	63	1	typical angina	Cleveland	145.0	233.0	
1	67	1	asymptomatic	Cleveland	160.0	286.0	
2	67	1	asymptomatic	Cleveland	120.0	229.0	
3	37	1	non-anginal	Cleveland	130.0	250.0	
4	41	0	atypical angina	Cleveland	130.0	204.0	

In [45]:
```
X = data_1.drop('target', axis=1)
y = data_1['target']
```

In [46]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

In [47]:
```
X.dtypes
```

Out[47]:
```
age                          int64
sex                          int64
chest_pain_type             object
country                     object
resting_blood_pressure     float64
cholesterol                float64
fasting_blood_sugar          int64
restecg                     object
max_heart_rate_achieved    float64
exercise_induced_angina      int64
st_depression              float64
st_slope_type               object
num_major_vessels          float64
thalassemia_type            object
dtype: object
```

In [48]:
```
le = LabelEncoder()

categorical_cols = X.select_dtypes(include='object').columns

for col in categorical_cols:
    X[col] = le.fit_transform(X[col])
```

In [49]:
```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [50]:
```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

In [51]:
```
model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)
```

Out[51]:

▾ LogisticRegression

▶ Parameters

In [52]:
```
y_pred = model.predict(X_test)
```

In [53]:
```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.782608695652174
```

In [54]:
```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[62 13]
 [27 82]]
```

In [55]:
```
from sklearn.metrics import roc_auc_score

y_prob = model.predict_proba(X_test)[:, 1]
roc_auc = roc_auc_score(y_test, y_prob)
print("ROC-AUC Score:", roc_auc)
```

```
ROC-AUC Score: 0.8707033639143732
```